

C51 / 8051 开卷考题解与速查资料

2026 年 6 月 24 日

目录

1 说明	5
2 习题 3: 汇编语言程序设计	6
第 1 题: MCS-51 寻址方式与存储空间区别	6
第 2 题: MCS-51 指令系统分类	6
第 3 题: 程序段执行结果分析	6
第 4 题: 两个 16 位数相减	7
第 5 题: 在 30H 到 50H 中查找 34H	7
第 6 题: 统计字节中 0 的个数	8
第 7 题: 压缩 BCD 码转 ASCII	8
第 8 题: 连续 20 个单元求平均值	8
第 9 题: 32 个有符号数中负数取补	9
第 10 题: 20 个无符号数按降序排序	10
3 习题 4: C51 程序设计	11
第 1 题: C51 语言特点	11
第 2 题: C51 支持的数据类型	11
第 3 题: 特殊存储类型变量定义	11
第 4 题: 8255 绝对地址变量定义	11
第 5 题: 十六进制字符转 ASCII	12
第 6 题: 整型变量左移 4 位	12
第 7 题: 一维数组初始化并求和	12
第 8 题: 求 n 的阶乘	12
第 9 题: 片外 RAM 两单元内容交换	13
第 10 题: 片内 RAM 30H 到 40H 升序排序	13
第 11 题: C 与汇编混合输出 90% 占空比方波	14
4 习题 6: 中断、定时器、串口	16
第 1 题: AT89S 并行口资源与功能	16
第 2 题: 声光报警器设计	16
第 3 题: 内部中断源与入口地址	17
第 4 题: 外部中断 0 事件计数	17
第 5 题: T0 与 T1 的工作方式分类	18
第 6 题: T0 定时 250us 产生 1s 方波	18
第 7 题: T2 定时 50ms 实现时钟	19
第 8 题: 串口的 4 种工作方式	20
第 9 题: 单片机多机通信原理	20
第 10 题: 4800bps 串口发送字符串	20
第 11 题: 两机 1200bps 块数据串口通信	22

5 习题 7: 存储器扩展技术	24
第 1 题: 程序存储器与数据存储器为何不冲突	24
第 2 题: AT89 扩展系统总线构建方法	24
第 3 题: 译码法与线选法比较	24
第 4 题: MOVX 访问 4000H 结果分析	24
第 5 题: 单片机存储器的主要功能	24
第 6 题: 外部 4000H 到 40FFH 清零	25
第 7 题: 地址线数量与存储容量计算	25
第 8 题: 区分片外程序与数据存储器	25
第 9 题: 2764 与 6264 扩展及地址分析	25
第 10 题: 2001H 与 2002H 数据拼接成 1 字节	26
6 习题 8: 接口扩展技术	27
第 1 题: I/O 接口功能与 I/O 端口区别	27
第 2 题: I/O 端口编址方式	27
第 3 题: I/O 数据传送方式	27
第 4 题: 8255A 方式 1 选通输入过程	27
第 5 题: 8255A 初始化控制字设计	27
第 6 题: 键盘的 3 种工作方式	28
第 7 题: 非编码键盘原理与去抖	28
第 8 题: 矩阵键盘按键识别原理	29
第 9 题: LED 静态与动态显示比较	29
第 10 题: LCD 显示器与键盘接口设计	29
第 11 题: A/D 转换器主要技术指标	30
第 12 题: 常用 A/D 转换器类型与特点	31
第 13 题: 逐次逼近式 A/D 转换原理	31
第 14 题: D/A 转换器主要技术指标	31
第 15 题: DAC0832 控制信号及作用	31
第 16 题: ADC0809 接口与定时采样	32
7 习题 9: 串行接口应用	34
第 1 题: 常用串行总线及特点	34
第 2 题: TLC549 数据采集与 LED 显示	34
第 3 题: TLC5615 波形发生器设计	35
A 附录 A: C51 / 8051 汇编常用指令速查表	39
B 附录 B: 寄存器位定义与中断定时器配置速查	41
1. 常用寄存器位定义与含义: IE、IP、TCON、TMOD、SCON、T2CON 速查	41
2. 如何配置中断: 通用流程与示例	42
3. 定时器 T0/T1 的四种模式: 模式 0、1、2、3 的特点与用途	43
4. 如何计算定时器参数: 计数节拍、计数值与初值公式	43
5. 如何把参数配置到定时器里: 以 12MHz、1ms 定时为例	44

6. 四种模式的典型配置模板：T0/T1 常用初始化写法	45
7. 考试中最常用的结论：最容易直接套用的经验	45

1 说明

本文根据题图中的 习题 3、4、6、7、8、9 整理。使用约定如下：

- 习题 3 中的程序题按 MCS-51 汇编给出。
- 其他程序题若题目未特别说明，则按 C51 给出。
- 代码以 REG52.H、传统 12T 8051/AT89S 系列为默认背景；若题目给出了晶振频率，则按题目参数计算。
- 对“画电路图”的题，这里整理成 连线表 + 地址/控制说明 + 程序框架，便于开卷考试时快速抄写。

2 习题 3: 汇编语言程序设计

第 1 题

MCS-51 常见寻址方式可归纳为:

1. 立即寻址: 操作数直接写在指令中, 如 `MOV A, #55H`。数据来自程序存储器中的指令字节。
2. 寄存器寻址: 操作数在寄存器中, 如 `MOV A, R0`。可用寄存器主要有 `R0~R7`、`A`、`B`、`DPTR` 等。
3. 直接寻址: 指令给出内部 RAM 或 SFR 的直接地址, 如 `MOV A, 30H`、`MOV P1, A`。
4. 寄存器间接寻址: 地址在寄存器中, 如 `MOV A, @R0`、`MOVX A, @DPTR`。其中 `@R0/@R1` 常用于片内 RAM, `@DPTR` 常用于片外数据存储器。
5. 变址寻址: 以 `A+DPTR` 或 `A+PC` 形成地址访问程序存储器查表, 如 `MOVC A, @A+DPTR`。
6. 位寻址: 直接访问位地址区或可位寻址 SFR 位, 如 `SETB P1.0`、`CLR PSW.5`。
7. 相对寻址: 用于短转移, 位移范围为 $-128 \sim +127$, 如 `SJMP LOOP`、`JNZ NEXT`。

对应存储空间差别:

- 立即寻址: 数据在指令本身中, 不再到 RAM 中取数。
- 寄存器寻址: 数据在 CPU 内部寄存器, 速度最快。
- 直接寻址: 可访问片内 RAM 低 128 字节和 SFR。
- 寄存器间接寻址: 可灵活访问 RAM, 也可通过 `MOVX` 访问片外 RAM。
- 变址寻址: 专门面向程序存储器查表。
- 位寻址: 只针对位地址区 (`20H~2FH`) 和可位寻址 SFR。
- 相对寻址: 本质上是程序转移地址形成方式, 不用于普通数据读取。

第 2 题

MCS-51 指令系统按功能可分为 5 类:

1. 数据传送类: `MOV`、`MOVX`、`MOVC`、`PUSH`、`POP`、`XCH` 等。
2. 算术运算类: `ADD`、`ADDC`、`SUBB`、`INC`、`DEC`、`MUL`、`DIV` 等。
3. 逻辑运算类: `ANL`、`ORL`、`XRL`、`CLR`、`CPL`、`RL`、`RR`、`SWAP` 等。
4. 控制转移类: `SJMP`、`LJMP`、`AJMP`、`JZ`、`JNZ`、`CJNE`、`DJNZ`、`CALL`、`RET` 等。
5. 位操作类: `SETB`、`CLR bit`、`CPL bit`、`JB`、`JNB`、`JBC`、`MOV C, bit` 等。

第 3 题

已知 $(50H) = 60H$, 程序段如下:

```
MOV A, 50H
MOV R0, A
MOV A, #00H
MOV @R0, A
MOV A, 3BH
MOV 61H, #60H
MOV 62H, A
```

逐句分析:

1. `MOV A, 50H` 后: $A=60H$
2. `MOV R0, A` 后: $R0=60H$

- 3. MOV A,#00H 后: A=00H
- 4. MOV @R0,A 后: (60H)=00H
- 5. MOV A,3BH 后: A=(3BH)
- 6. MOV 61H,#60H 后: (61H)=60H
- 7. MOV 62H,A 后: (62H)=(3BH)

最终结果:

量	内容
A	(3BH)
R0	60H
(60H)	00H
(61H)	60H
(62H)	(3BH)

说明: 题目只给出 (50H) = 60H, 未给出 (3BH) 初值, 因此 A 与 62H 只能写成符号结果。

第 4 题

30H/31H 构成第一个 16 位数, 32H/33H 构成第二个 16 位数, 完成减法并把结果送入 40H/41H。

```
CLR C
MOV A,31H
SUBB A,33H
MOV 41H,A

MOV A,30H
SUBB A,32H
MOV 40H,A
```

思路: 先减低字节, 再带借位减高字节。

第 5 题

查找 30H~50H 单元中是否有数据 34H, 若有则 F0=1, 否则 F0=0。

```
CLR F0
MOV R0,#30H
MOV R2,#21H

LOOP5: MOV A,@R0
      CJNE A,#34H,NEXT5
      SETB F0
      SJMP DONE5

NEXT5: INC R0
      DJNZ R2,LOOP5

DONE5:
```

说明: $30H$ 到 $50H$ 为含端点区间, 共 $50H - 30H + 1 = 21H$ 个单元。

第 6 题

求 $40H$ 单元内容中二进制位“0”的个数, 结果存入 $50H$ 。

```
MOV A, 40H
MOV R2, #08
MOV R3, #00

LOOP6: JNB ACC.0, INC0
        SJMP NEXT6

INC0:   INC R3

NEXT6:  RR A
        DJNZ R2, LOOP6
        MOV 50H, R3
```

第 7 题

$30H$ 开始的 5 个单元中存放 5 个压缩 BCD 码, 将其转换成 ASCII 码后存入 $40H$ 开始的单元。

```
MOV R0, #30H
MOV R1, #40H
MOV R2, #05

LOOP7: MOV A, @R0
        ANL A, #0F0H
        SWAP A
        ORL A, #30H
        MOV @R1, A
        INC R1

        MOV A, @R0
        ANL A, #0FH
        ORL A, #30H
        MOV @R1, A
        INC R1

        INC R0
        DJNZ R2, LOOP7
```

说明: 每个压缩 BCD 字节拆成高 4 位和低 4 位两个 ASCII 字符, 因此输出共 10 字节。

第 8 题

求片内 RAM 中从 $30H$ 开始连续 20 个单元内容的平均值, 存入 $60H$ 。

```
MOV R0, #30H
MOV R2, #20
MOV 70H, #00H
```



```

        MOV    71H,#00H

SUM8:   MOV    A,70H
        ADD    A,@R0
        MOV    70H,A
        JNC    NOC8
        INC    71H
NOC8:   INC    R0
        DJNZ   R2,SUM8

        MOV    72H,#00H

DIV8:   MOV    A,71H
        JNZ    SUB20
        MOV    A,70H
        CLR    C
        SUBB   A,#14H
        JC     DONE8

SUB20:   MOV    A,70H
        CLR    C
        SUBB   A,#14H
        MOV    70H,A
        MOV    A,71H
        SUBB   A,#00H
        MOV    71H,A
        INC    72H
        SJMP   DIV8

DONE8:   MOV    60H,72H

```

说明：20 个 8 位无符号数求和可能超过 255，所以先做 16 位累加，再用减 20 法求整数平均值。

第 9 题

检查从 30H 开始的 32 个数：若为正数或 0，则不变；若为负数，则取补后送回。

```

        MOV    R0,#30H
        MOV    R2,#32

LOOP9:  MOV    A,@R0
        JB     ACC.7,NEG9
        SJMP   NEXT9

NEG9:   CPL    A
        INC    A
        MOV    @R0,A

NEXT9:  INC    R0
        DJNZ   R2,LOOP9

```

说明：这里把“取补”理解为对负数求其二进制补码的绝对值形式，即 CPL 后再 INC。

第 10 题

将 30H 开始的 20 个 8 位无符号数按从大到小排序，并从 20H 单元开始存放。

```
MOV R0,#30H
MOV R1,#20H
MOV R2,#20

COPY10: MOV A,@R0
MOV @R1,A
INC R0
INC R1
DJNZ R2,COPY10

MOV R7,#19

OUTER10:
MOV R0,#20H
MOV R6,#19

INNER10:
MOV A,@R0
INC R0
CJNE A,@R0,CMP10
SJMP NEXT10

CMP10: JC SWAP10
SJMP NEXT10

SWAP10: MOV R5,@R0
MOV @R0,A
DEC R0
MOV A,R5
MOV @R0,A
INC R0

NEXT10: DJNZ R6,INNER10
DJNZ R7,OUTER10
```

3 习题 4: C51 程序设计

第 1 题

C51 语言的主要特点:

- 以 ANSI C 为基础, 语法接近标准 C, 上手方便。
- 针对 8051 扩充了 `bit`、`sbit`、`sfr`、`sfr16`、`data`、`idata`、`xdata`、`code`、`bdata` 等关键字。
- 支持单片机多存储空间编程, 便于区分片内 RAM、片外 RAM、程序存储器。
- 直接支持中断函数定义, 适合硬件控制。
- 支持 C 与汇编混合编程, 便于兼顾开发效率与执行效率。

第 2 题

C51 支持的数据类型可分为:

- 基本类型: `char`、`int`、`short`、`long`、`float`、`bit`、`void`
- 构造类型: 数组、指针、结构体、联合体、枚举
- 专用硬件类型: `sbit`、`sfr`、`sfr16`

第 3 题

分别定义下列变量:

```
unsigned char data x;  
unsigned char bdata y;  
sbit key_in    = y^0;  
sbit key_up    = y^1;  
sbit key_down  = y^2;  
int xdata x_ext _at_ 0x4000;  
sfr SCON = 0x98;
```

说明:

- `data`: 片内 RAM 直接寻址区。
- `bdata`: 片内 RAM 位寻址区。
- `sbit`: 把某一位定义为位变量。
- `xdata`: 片外 RAM。
- `sfr`: 特殊功能寄存器。

第 4 题

将外部 8255 的 PA、PB、PC、控制口定义为绝对地址 7FFCH、7FFDH、7FFEh、7FFFh 的绝对地址字节变量:

```
unsigned char xdata PA8255 _at_ 0x7FFC;  
unsigned char xdata PB8255 _at_ 0x7FFD;  
unsigned char xdata PC8255 _at_ 0x7FFE;  
unsigned char xdata CTRL8255 _at_ 0x7FFF;
```

第 5 题

将 0 ~ 9、A ~ F 转换成对应 ASCII 码:

```
unsigned char hex_to_ascii(unsigned char x)
{
    x &= 0x0F;
    if (x < 10)
        return x + '0';
    else
        return x - 10 + 'A';
}
```

第 6 题

整型变量 a 左移 4 位:

```
void shift_left_4(void)
{
    int a = 0x0123;
    a <<= 4;
}
```

第 7 题

定义一个 20 元素一维数组, 放在片内 RAM 中, 数组元素值等于其序号, 并求和。

```
#include <REG52.H>

unsigned char data arr[20];
unsigned int sum;

void main(void)
{
    unsigned char i;
    sum = 0;
    for (i = 0; i < 20; i++)
    {
        arr[i] = i;
        sum += arr[i];
    }
    while (1);
}
```

第 8 题

求 $n!$ 的 C51 程序:

```
#include <REG52.H>

unsigned long fact(unsigned char n)
```

```
{
    unsigned char i;
    unsigned long r = 1;
    for (i = 1; i <= n; i++)
        r *= i;
    return r;
}

void main(void)
{
    unsigned long ans;
    ans = fact(5);
    while (1);
}
```

说明：若 n 较大，需注意 unsigned long 溢出。

第 9 题

完成片外 RAM 的 000EH 与 000FH 单元内容交换：

```
#include <REG52.H>

void main(void)
{
    unsigned char xdata *p = (unsigned char xdata *)0x000E;
    unsigned char t;

    t = p[0];
    p[0] = p[1];
    p[1] = t;

    while (1);
}
```

第 10 题

对片内 RAM 中 30H~40H 的数据按从小到大排序。

```
#include <REG52.H>

unsigned char data buf[17] _at_ 0x30;

void main(void)
{
    unsigned char i, j, t;
    for (i = 0; i < 16; i++)
    {
        for (j = 0; j < 16 - i; j++)
        {
            if (buf[j] > buf[j + 1])
```

```

        {
            t = buf[j];
            buf[j] = buf[j + 1];
            buf[j + 1] = t;
        }
    }
}
while (1);
}

```

说明：30H 到 40H 为含端点区间，共 17 个数据。

第 11 题

用 C51 与汇编混合编程，使 P1.0 输出占空比 90%、周期 10ms 的方波。

C 文件：

```

#include <REG52.H>

extern void delay1ms(void);
sbit WAVE = P1^0;

void delay_ms(unsigned char ms)
{
    while (ms--)
        delay1ms();
}

void main(void)
{
    while (1)
    {
        WAVE = 1;
        delay_ms(9);
        WAVE = 0;
        delay_ms(1);
    }
}

```

汇编延时子程序：

```

PUBLIC _delay1ms

?PR?_delay1ms?DLY SEGMENT CODE
    RSEG    ?PR?_delay1ms?DLY

_delay1ms:
    MOV     R7,#250
L1:    MOV     R6,#248
L2:    DJNZ    R6,L2
        DJNZ    R7,L1
    RET

```

END

4 习题 6: 中断、定时器、串口

第 1 题

AT89S 系列单片机并行口资源及功能:

1. P0 口: 8 位双向口, 开漏结构; 扩展时可作复用低 8 位地址/数据总线 AD0~AD7。
2. P1 口: 8 位准双向通用 I/O 口, 一般无复用功能, 常用于键盘、LED、继电器等。
3. P2 口: 8 位准双向口; 扩展存储器时输出高 8 位地址 A8~A15。
4. P3 口: 8 位准双向口, 同时具有第二功能:
 - P3.0 / P3.1: RXD / TXD
 - P3.2 / P3.3: INT0 / INT1
 - P3.4 / P3.5: T0 / T1
 - P3.6 / P3.7: WR / RD

第 2 题

设计一个声光报警器: 设备正常运行时绿灯亮; 非正常运行时红灯闪烁且报警器持续发声。

接口假定:

- P1.0 接绿灯
- P1.1 接红灯
- P1.2 接蜂鸣器
- P3.2 接状态输入, 1 表示异常, 0 表示正常

```
#include <REG52.H>

sbit LED_G = P1^0;
sbit LED_R = P1^1;
sbit BUZZER = P1^2;
sbit ALM_IN = P3^2;

void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    for (i = 0; i < ms; i++)
        for (j = 0; j < 110; j++);
}

void main(void)
{
    while (1)
    {
        if (ALM_IN == 0)
        {
            LED_G = 1;
            LED_R = 0;
            BUZZER = 0;
        }
        else
        {
            LED_G = 0;
            LED_R = 1;
            BUZZER = 1;
        }
    }
}
```



```
        LED_G = 0;
        LED_R = !LED_R;
        BUZZER = 1;
        delay_ms(200);
    }
}
```

第 3 题

AT89S52 常见内部中断源与入口地址:

中断源	中断号	入口地址
外部中断 0	0	0003H
定时器 0 溢出	1	000BH
外部中断 1	2	0013H
定时器 1 溢出	3	001BH
串口中断	4	0023H
定时器 2 溢出 / T2EX	5	002BH

第 4 题

设计一个外部事件中断计数器, 采用外部中断 0 边沿触发方式对外部中断事件计数。

```
#include <REG52.H>

volatile unsigned int ext0_cnt = 0;

void int0_isr(void) interrupt 0
{
    ext0_cnt++;
}

void main(void)
{
    IT0 = 1;
    EX0 = 1;
    EA = 1;

    while (1)
    {
        P1 = (unsigned char)ext0_cnt;
    }
}
```

第 5 题

按计数器结构划分, T0、T1 有 4 种工作方式:

1. 方式 0: 13 位定时/计数器
2. 方式 1: 16 位定时/计数器
3. 方式 2: 8 位自动重装
4. 方式 3: 定时器 0 分成两个独立 8 位计数器

按计数脉冲来源划分, T0、T1 有 2 种工作方式:

1. 定时方式: 脉冲来自片内机器周期时钟。
2. 计数方式: 脉冲来自外部引脚 T0/P3.4 或 T1/P3.5。

第 6 题

12MHz 晶振下, T0 产生 $250\mu s$ 定时中断, 使 P3.4 输出周期为 1s 的方波。

计算:

- 12MHz 下机器周期为 $1\mu s$
- 250 个计数对应 $250\mu s$
- 方式 1 初值: $65536 - 250 = 65286 = FF06H$
- 方波半周期为 0.5s, 需要中断 $500000/250 = 2000$ 次翻转一次

```
#include <REG52.H>

sbit WAVE = P3^4;
volatile unsigned int t0_cnt = 0;

void timer0_isr(void) interrupt 1
{
    TH0 = 0xFF;
    TL0 = 0x06;

    t0_cnt++;
    if (t0_cnt >= 2000)
    {
        t0_cnt = 0;
        WAVE = !WAVE;
    }
}

void main(void)
{
    TMOD &= 0xF0;
    TMOD |= 0x01;
    TH0 = 0xFF;
    TL0 = 0x06;
    ETO = 1;
    EA = 1;
    TR0 = 1;

    while (1);
}
```

```
}

```

第 7 题

12MHz 晶振下, 用定时器 2 产生 50ms 定时, 每隔 1s 对 30H~32H (时、分、秒) 计时。

计算:

- $50ms = 50000\mu s$
- 重装值: $65536 - 50000 = 15536 = 3CB0H$
- 20 次 50ms = 1s

```
#include <REG52.H>

unsigned char data clock_buf[3] _at_ 0x30;
volatile unsigned char tick50ms = 0;

void timer2_isr(void) interrupt 5
{
    TF2 = 0;
    tick50ms++;

    if (tick50ms >= 20)
    {
        tick50ms = 0;
        clock_buf[2]++;
        if (clock_buf[2] >= 60)
        {
            clock_buf[2] = 0;
            clock_buf[1]++;
            if (clock_buf[1] >= 60)
            {
                clock_buf[1] = 0;
                clock_buf[0]++;
                if (clock_buf[0] >= 24)
                    clock_buf[0] = 0;
            }
        }
    }
}

void main(void)
{
    clock_buf[0] = 0;
    clock_buf[1] = 0;
    clock_buf[2] = 0;

    T2CON = 0x00;
    RCAP2H = 0x3C;
    RCAP2L = 0xB0;
    TH2 = 0x3C;
    TL2 = 0xB0;
}
```

```
ET2    = 1;
EA      = 1;
TR2     = 1;

while (1);
}
```

第 8 题

AT89S 系列单片机串行口有 4 种工作方式:

方式	特点	数据位	波特率
方式 0	同步移位寄存器方式	8 位	固定, 为 $f_{osc}/12$
方式 1	8 位 UART	10 位帧 (1 起始 + 8 数据 + 1 停止)	可变
方式 2	9 位 UART	11 位帧	固定, 可为 $f_{osc}/64$ 或 $f_{osc}/32$
方式 3	9 位 UART	11 位帧	可变

第 9 题

AT89S 系列单片机多机通信原理:

- 一般采用串口方式 2 或方式 3 的 9 位通信。
- 第 9 位可作为“地址/数据”标志位。
- 主机先发送地址帧, 令 $TB8=1$ 。
- 各从机在 $SM2=1$ 时只接收地址帧; 若地址匹配, 则该从机清 $SM2$, 继续接收后续数据帧。
- 数据帧发送时 $TB8=0$, 只有被选中的从机参与接收, 其他从机忽略。

第 10 题

11.0592MHz 晶振, 串口方式 1, 波特率 4800bit/s, 发送字符串 "AT89S52 Micro computer"。分别给出查询方式和中断方式。

参数计算:

$$TH1 = 256 - \frac{11059200}{384 \times 4800} = 256 - 6 = FAH$$

查询方式:

```
#include <REG52.H>

char code msg[] = "AT89S52 Micro computer";

void uart_init(void)
{
    TMOD &= 0x0F;
    TMOD |= 0x20;
    SCON = 0x50;
```

```

    TH1 = 0xFA;
    TL1 = 0xFA;
    TR1 = 1;
}

void uart_send_char(char c)
{
    SBUF = c;
    while (TI == 0);
    TI = 0;
}

void uart_send_str(char *s)
{
    while (*s)
        uart_send_char(*s++);
}

void main(void)
{
    uart_init();
    uart_send_str(msg);
    while (1);
}

```

中断方式:

```

#include <REG52.H>

char code msg[] = "AT89S52 Micro computer";
volatile unsigned char tx_idx = 0;

void uart_init(void)
{
    TMOD &= 0x0F;
    TMOD |= 0x20;
    SCON = 0x50;
    TH1 = 0xFA;
    TL1 = 0xFA;
    ES = 1;
    EA = 1;
    TR1 = 1;
}

void serial_isr(void) interrupt 4
{
    if (TI)
    {
        TI = 0;
        if (msg[tx_idx] != '\0')
            SBUF = msg[tx_idx++];
    }
}

```

```

    if (RI)
        RI = 0;
}

void main(void)
{
    uart_init();
    tx_idx = 1;
    SBUF = msg[0];
    while (1);
}

```

第 11 题

0# 单片机以 1200bit/s 从串口发送内部 RAM 20H~30H 的数据块；1# 单片机接收并保存。

参数计算：

$$TH1 = 256 - \frac{11059200}{384 \times 1200} = 256 - 24 = E8H$$

发送机程序：

```

#include <REG52.H>

unsigned char data txbuf[17] _at_ 0x20;

void uart_init(void)
{
    TMOD &= 0x0F;
    TMOD |= 0x20;
    SCON = 0x50;
    TH1 = 0xE8;
    TL1 = 0xE8;
    TR1 = 1;
}

void uart_send(unsigned char dat)
{
    SBUF = dat;
    while (!TI);
    TI = 0;
}

void main(void)
{
    unsigned char i;
    uart_init();
    for (i = 0; i < 17; i++)
        uart_send(txbuf[i]);
    while (1);
}

```

接收机程序:

```
#include <REG52.H>

unsigned char data rxbuf[17] _at_ 0x20;

void uart_init(void)
{
    TMOD &= 0x0F;
    TMOD |= 0x20;
    SCON = 0x50;
    TH1 = 0xE8;
    TL1 = 0xE8;
    TR1 = 1;
}

unsigned char uart_rcv(void)
{
    while (!RI);
    RI = 0;
    return SBUF;
}

void main(void)
{
    unsigned char i;
    uart_init();
    for (i = 0; i < 17; i++)
        rxbuf[i] = uart_rcv();
    while (1);
}
```

5 习题 7: 存储器扩展技术

第 1 题

在单片机扩展系统中，程序存储器和数据存储器虽然共用 16 位地址总线和 8 位数据总线，但不会发生空间冲突，原因是：

- MCS-51 采用 程序存储器空间与数据存储器空间分离的结构；
- 访问程序存储器使用 PSEN 控制；
- 访问外部数据存储器使用 RD/WR 控制；
- 即使地址数值相同，只要控制信号不同，访问的就不是同一类存储器。

第 2 题

AT89 系列单片机扩展系统总线构建方法：

1. P0 口复用为低 8 位地址/数据总线 AD0~AD7；
2. 通过锁存器（如 74LS373）在 ALE 作用下锁存低 8 位地址；
3. P2 口输出高 8 位地址 A8~A15；
4. PSEN 控制程序存储器读；
5. RD/WR 控制片外数据存储器或 I/O 读写；
6. 通过线选法或译码法产生片选信号。

第 3 题

译码法与线选法比较：

方法	优点	缺点
译码法	地址分配唯一，不易出现地址镜像，空间利用规范	需要译码器，硬件略复杂
线选法	电路简单、成本低、实现快	容易出现地址重叠/镜像，地址利用不充分

第 4 题

设外部数据存储器 4000H 单元内容为 80H，执行下列指令后累加器 A 的内容为多少：

```
MOV P2, #40H
MOV R0, #00H
MOVX A, @R0
```

分析：MOVX A,@R0 访问片外数据存储器时，低地址来自 R0，高地址来自 P2，因此访问地址为 4000H。

$$A = (4000H) = 80H$$

第 5 题

单片机存储器的主要功能是存储 程序和 数据。

第 6 题

将外部数据存储器 4000H~40FFH 单元全部清零:

```
#include <REG52.H>

void main(void)
{
    unsigned int i;
    unsigned char xdata *p = (unsigned char xdata *)0x4000;

    for (i = 0; i < 256; i++)
        p[i] = 0x00;

    while (1);
}
```

第 7 题

11 根地址线可选:

$$2^{11} = 2048$$

个存储单元。

16KB 存储单元需要地址线数:

$$16KB = 2^{14}B$$

所以需要 14 根地址线。

第 8 题

区分 MCS-51 片外程序存储器和片外数据存储器的最可靠方法是:

D: 看其是连到 RD 还是 $PSEN$

第 9 题

用一片 EPROM 2764 和一片 SRAM 6264 连接到 AT89 单片机, 写出地址范围并判断是否有地址码重叠。

一种规范的译码连接方案如下:

1. P0 经过 74LS373 锁存后得到 A0~A7;
2. P2 直接提供 A8~A15;
3. 用 74LS138 对高位地址译码, 取 A15~A13 = 000 时选通一页 8KB 空间;
4. 2764 的 A0~A12 接总线地址线, /OE 接 PSEN, /CE 接译码输出;
5. 6264 的 A0~A12 接总线地址线, /OE 接 RD, /WE 接 WR, /CE 接译码输出。

地址范围:

- 外部程序存储器 2764: 0000H~1FFFH (代码空间)

- 外部数据存储器 6264: 0000H~1FFFFH (数据空间)

是否重叠:

- 从数值地址看, 两者范围相同;
- 但程序空间与数据空间由 PSEN 和 RD/WR 区分, 因此 功能上不冲突;
- 若采用上述译码法, 则 无地址镜像现象;
- 若简单线选、片选线不参与高位译码, 则可能在整个 64KB 空间中出现周期性镜像。

第 10 题

将外部数据区 2001H、2002H 中两个数据按顺序拼接为 1 个字节。例如 05H 和 06H 拼为 56H, 结果存回 2002H。

```
#include <REG52.H>

void main(void)
{
    unsigned char xdata *p1 = (unsigned char xdata *)0x2001;
    unsigned char xdata *p2 = (unsigned char xdata *)0x2002;
    unsigned char a, b, c;

    a = *p1 & 0x0F;
    b = *p2 & 0x0F;
    c = (a << 4) | b;
    *p2 = c;

    while (1);
}
```

6 习题 8: 接口扩展技术

第 1 题

I/O 接口的功能:

- 实现 CPU 与外设之间的数据缓冲与隔离;
- 完成电平变换、驱动能力匹配;
- 提供端口地址译码;
- 产生读写控制和时序配合;
- 在需要时提供中断、握手、锁存等功能。

I/O 接口与 I/O 端口的区别:

- I/O 端口是 CPU 可访问的寄存器或引脚集合;
- I/O 接口是更完整的电路单元, 包含端口、缓冲、译码、控制逻辑等。

第 2 题

常用 I/O 端口编址方式有两种:

1. 统一编址 (存储器映射 I/O): I/O 口和数据存储器统一编址, 优点是指令系统统一, 缺点是占用存储器地址空间。
2. 独立编址 (专用 I/O 编址): I/O 口有独立地址空间, 优点是不占用存储器地址, 缺点是需要专门 I/O 指令。

AT89 系列扩展 I/O 一般采用 统一编址, 即把外设映射到 xdata 空间, 通过 MOVX 访问。

第 3 题

I/O 数据传送方式常见有:

1. 程序直接控制方式: 包括无条件传送、查询传送; 适用于低速、实时性要求不高的外设。
2. 中断传送方式: 适用于随机事件或需要及时响应的外设, 如按键、串口接收。
3. DMA 方式: 适用于大批量高速数据传输; 经典 8051 通常不内置 DMA, 但在体系结构上是常见方式。

第 4 题

8255A 的 A 口工作在方式 1 选通输入时, 其工作过程为:

1. 外设先把数据送到 A 口输入线上。
2. 外设发出选通信号 STB, 8255 锁存输入数据。
3. 8255 置位输入缓冲满信号 IBF。
4. 若中断允许 INTE=1, 则 8255 置位中断请求 INTR, 通知 CPU 读取数据。
5. CPU 读 A 口后, IBF 和 INTR 被清除, 等待下一次输入。

第 5 题

要求: A 口方式 0 输出, B 口方式 1 输入, C 口 PC7 为输入, PC1 为输出。

结论: 该要求本身存在冲突。

- 当 B 口工作在方式 1 输入时, PC0~PC2 被 8255 占用为握手线, 其中 PC1 不再能作为普通输出位使用。
- 同时, 方式设置字只能按 上半个 C 口/下半个 C 口配置方向, 不能只把 PC7 单独定义为输入而不影响 PC4~PC6。

若按尽可能接近题意的可实现配置处理:

- A 口: 方式 0 输出
- B 口: 方式 1 输入
- C 口上半字节: 输入
- C 口下半字节: 输出 (但 PC0~PC2 实际会被 B 口方式 1 握手占用)

则方式控制字可写为:

$$10001110B = 8EH$$

初始化程序:

```
#include <REG52.H>

unsigned char xdata CTRL8255 _at_ 0x7FFF;

void main(void)
{
    CTRL8255 = 0x8E;
    while (1);
}
```

第 6 题

键盘常见 3 种工作方式:

1. 程序扫描方式: CPU 循环扫描键盘状态, 实现简单, 但占用 CPU 时间。
2. 定时扫描方式: 用定时器周期性扫描, 时间规律, 适合周期性检测。
3. 中断方式: 按键动作触发中断, 响应快、CPU 空闲率高, 但硬件/软件略复杂。

第 7 题

非编码键盘的工作原理:

- 每个按键直接对应一个 I/O 输入位;
- CPU 读取端口状态即可判断哪个按键被按下。

去抖方法:

- 检测到按下后延时 10~20ms, 再读一次;
- 两次一致才确认有效按键。

判断按键释放:

- 检测到按下后, 可继续等待该键恢复到空闲电平;
- 再延时去抖并复查一次, 确认其已释放。

第 8 题

矩阵式键盘按键识别原理:

- 把键盘按“行线”和“列线”组成矩阵;
- 扫描时逐行输出低电平, 其余行为高电平;
- 同时读取列线状态;
- 若某列被读到低电平, 则“当前行 + 该列”交点按键按下。

第 9 题

LED 静态显示与动态显示比较:

方式	优点	缺点
静态显示	电路直观、程序简单、亮度稳定	占 I/O 口多, 硬件成本高
动态显示	节省 I/O 口和驱动器件	程序需循环扫描, 刷新不当会闪烁

第 10 题

设计一个 LCD 显示器/键盘接口电路。

参考方案:

- LCD 采用 1602;
- LCD 数据口 D0~D7 接 P0;
- RS、RW、E 分别接 P2.5、P2.6、P2.7;
- 键盘采用 4 × 4 矩阵键盘;
- 行线接 P1.0~P1.3, 列线接 P1.4~P1.7。

程序框架:

```
#include <REG52.H>

sbit LCD_RS = P2^5;
sbit LCD_RW = P2^6;
sbit LCD_EN = P2^7;

void delay_ms(unsigned int ms)
{
    unsigned int i, j;
    for (i = 0; i < ms; i++)
        for (j = 0; j < 110; j++);
}

void lcd_write_cmd(unsigned char cmd)
{
    LCD_RS = 0;
    LCD_RW = 0;
    P0 = cmd;
    LCD_EN = 1;
```

```
LCD_EN = 0;
delay_ms(2);
}

void lcd_write_dat(unsigned char dat)
{
    LCD_RS = 1;
    LCD_RW = 0;
    P0 = dat;
    LCD_EN = 1;
    LCD_EN = 0;
    delay_ms(2);
}

unsigned char key_scan(void)
{
    unsigned char row, col, val;
    P1 = 0xF0;
    if ((P1 & 0xF0) != 0xF0)
    {
        delay_ms(10);
        if ((P1 & 0xF0) != 0xF0)
        {
            for (row = 0; row < 4; row++)
            {
                P1 = ~(1 << row);
                val = P1 & 0xF0;
                if (val != 0xF0)
                {
                    for (col = 0; col < 4; col++)
                    {
                        if ((val & (0x10 << col)) == 0)
                            return row * 4 + col;
                    }
                }
            }
        }
    }
    return 0xFF;
}
```

第 11 题

A/D 转换器的主要技术指标:

- 分辨率
- 转换精度
- 转换时间 / 转换速度
- 输入范围
- 零点误差、满度误差

- 线性度 (INL、DNL)
- 单调性

第 12 题

常用 A/D 转换器类型及特点:

1. 逐次逼近型: 速度较快、精度较高、应用最广。
2. 双积分型: 抗干扰能力强、精度高, 但速度较慢。
3. 并行比较型 (闪速型): 速度最快, 但硬件复杂、成本高。
4. 跟踪型: 电路较简单, 适于连续变化信号。
5. Σ - Δ 型: 高分辨率、低速, 适于高精度测量。

第 13 题

逐次逼近式 A/D 工作原理:

1. 先把逐次逼近寄存器 (SAR) 的最高位置 1;
2. 经内部 DAC 产生一个试探电压;
3. 与输入模拟电压比较;
4. 若试探电压不大于输入电压, 则该位保留为 1; 否则清 0;
5. 再测试下一位, 直到最低位;
6. 全部位比较完成后, SAR 中的值即为转换结果。

第 14 题

D/A 转换器的主要技术指标:

- 分辨率
- 精度
- 建立时间
- 线性度
- 单调性
- 输出范围
- 零点漂移与温漂

第 15 题

DAC0832 与单片机连接时常用控制信号及其作用:

信号	作用
CS	片选信号, 决定 DAC0832 是否被访问
WR1	第一写信号, 把并行总线数据写入输入锁存器
WR2 / XFER	第二写/传送控制, 把输入锁存器数据送入 DAC 锁存器并更新模拟输出
ILE	输入锁存允许控制, 通常置高以允许写入

说明: 若系统不要求双缓冲同步更新, 可把部分控制线固定成简单工作方式。

第 16 题

在 AT89S52 系统中接入地址为 7FF8H~7FFFH 的 ADC0809, 写出硬件连接思路和定时采样通道 1 程序。

硬件连接思路:

- 用地址译码把 7FF8H~7FFFH 这一组 8 个地址选给 ADC0809。
- 把地址线 A0~A2 接到 ADC0809 的通道地址选择线 ADDA、ADDB、ADDC。
- 这样访问 7FF8H+n 时, 相当于选择通道 n 。
- 写该地址产生 ALE + START 启动脉冲; 读该地址时用 RD 使 OE 有效, 读回 8 位结果。
- EOC 可接到 P3.2 作为转换结束检测输入。

示例程序 (以下按 12MHz、T0 每 10ms 采样一次通道 1):

```
#include <REG52.H>

sbit ADC_EOC = P3^2;
volatile unsigned char adc_ch1_value = 0;

unsigned char adc0809_read(unsigned char ch)
{
    volatile unsigned char xdata *adc;
    adc = (unsigned char xdata *) (0x7FF8 | (ch & 0x07));

    *adc = 0x00;
    while (!ADC_EOC);
    return *adc;
}

void timer0_isr(void) interrupt 1
{
    TH0 = 0xD8;
    TL0 = 0xF0;
    adc_ch1_value = adc0809_read(1);
    P1 = adc_ch1_value;
}

void main(void)
{
    TMOD &= 0xF0;
    TMOD |= 0x01;
    TH0 = 0xD8;
    TL0 = 0xF0;
    ET0 = 1;
    EA = 1;
    TR0 = 1;

    while (1);
}
```


说明: 10ms 的方式 1 装载值按 12MHz 晶振计算为 $65536 - 10000 = 55536 = D8F0H$ 。

7 习题 9：串行接口应用

第 1 题

常用串行总线及特点：

总线	主要特点
UART / RS-232	点对点、异步通信、实现简单、距离较远，但线数较多、总线能力弱
SPI	全双工、速度快、主从结构、线数较多（一般 4 线）
I ² C	两线总线、支持多主多从、硬件简单，但速度一般低于 SPI
Microwire	与 SPI 类似，三线串行，常用于简单串行外设
1-Wire	单线通信、布线最简单，但速率低、协议时序较严格
CAN	抗干扰强、支持多节点仲裁、适合工业现场和汽车电子

第 2 题

用 AT89S51 和串行 A/D 转换器 TLC549 设计一个简单数据采集显示系统，把模拟电压的二进制值通过 P1 口 LED 显示。

硬件连接建议：

- TLC549 DATA OUT → P3.0
- TLC549 I/O CLOCK → P3.1
- TLC549 CS → P3.2
- P1.0~P1.7 → 8 个 LED（串限流电阻）
- REF+ 接 +5V，REF- 接地，ANALOG IN 接待测模拟电压

软件说明：

- TLC549 为串行输出 ADC，读取时通过时钟移出结果。
- 实际使用中通常第一次读数作“空读/预读”，第二次开始得到稳定结果。

```
#include <REG52.H>

sbit ADC_D0 = P3^0;
sbit ADC_CLK = P3^1;
sbit ADC_CS = P3^2;

void delay_us(void)
{
    _nop_();
}

unsigned char tlc549_read(void)
{
    unsigned char i;
    unsigned char dat = 0;
```

```
ADC_CS = 0;
for (i = 0; i < 8; i++)
{
    dat <<= 1;
    if (ADC_DO)
        dat |= 0x01;

    ADC_CLK = 1;
    delay_us();
    ADC_CLK = 0;
    delay_us();
}
ADC_CS = 1;
return dat;
}

void main(void)
{
    unsigned char value;
    ADC_CS = 1;
    ADC_CLK = 0;

    tlc549_read();

    while (1)
    {
        value = tlc549_read();
        P1 = value;
    }
}
```

第 3 题

用 AT89S51 和串行 D/A 转换器 TLC5615 设计一个简单波形发生器: S1 输出方波, S2 输出锯齿波, S3 输出三角波, S4 输出正弦波。

硬件连接建议:

- TLC5615 DIN → P3.0
- TLC5615 SCLK → P3.1
- TLC5615 CS → P3.2
- TLC5615 OUT → 示波器 / 运放缓冲 / RC 滤波
- REFIN 接基准电压 (如 2.5V)
- S1~S4 分别接 P1.0~P1.3, 低电平有效

程序说明:

- TLC5615 可采用 12 位串行写入: 10 位数据 + 2 个补零位。
- 发送时 CS 拉低, 数据按 MSB 先行送入, 最后 CS 上升沿更新 DAC 输出。

```
#include <REG52.H>
```

```
sbit DAC_DIN  = P3^0;
sbit DAC_CLK  = P3^1;
sbit DAC_CS   = P3^2;
sbit S1       = P1^0;
sbit S2       = P1^1;
sbit S3       = P1^2;
sbit S4       = P1^3;

unsigned int code sine_tab[32] = {
    512,612,707,793,866,923,962,982,
    982,962,923,866,793,707,612,512,
    412,317,231,158,101, 62, 42, 42,
    62,101,158,231,317,412,512,512
};

void short_delay(void)
{
    unsigned char i;
    for (i = 0; i < 80; i++);
}

void dac_write(unsigned int value)
{
    unsigned char i;
    unsigned int word = (value & 0x03FF) << 2;

    DAC_CS = 0;
    for (i = 0; i < 12; i++)
    {
        DAC_DIN = (word & 0x0800) ? 1 : 0;
        DAC_CLK = 1;
        DAC_CLK = 0;
        word <<= 1;
    }
    DAC_CS = 1;
}

void wave_square(void)
{
    dac_write(0);
    short_delay();
    dac_write(1023);
    short_delay();
}

void wave_sawtooth(void)
{
    unsigned int i;
    for (i = 0; i < 1024; i += 16)
    {
        dac_write(i);
    }
}
```

```
        short_delay();
    }
}

void wave_triangle(void)
{
    unsigned int i;
    for (i = 0; i < 1024; i += 16)
    {
        dac_write(i);
        short_delay();
    }
    for (i = 1023; i > 15; i -= 16)
    {
        dac_write(i);
        short_delay();
    }
}

void wave_sine(void)
{
    unsigned char i;
    for (i = 0; i < 32; i++)
    {
        dac_write(sine_tab[i]);
        short_delay();
    }
}

void main(void)
{
    DAC_CS = 1;
    DAC_CLK = 0;

    while (1)
    {
        if (S1 == 0)
            wave_square();
        else if (S2 == 0)
            wave_sawtooth();
        else if (S3 == 0)
            wave_triangle();
        else if (S4 == 0)
            wave_sine();
        else
            dac_write(0);
    }
}
```

Proteus 验证要点:

- 在 TLC5615 输出端接示波器;

- 依次按下 S1~S4, 观察方波、锯齿波、三角波、正弦波;
- 若正弦波阶梯感较明显, 可加 RC 平滑或增加查表点数。

A 附录 A: C51 / 8051 汇编常用指令速查表

指令	用法	举例	易错点
MOV	数据传送	MOV A, 30H	不能直接“内存到内存”; @Ri 间接寻址时 只能用 R0 或 R1; MOV 不能访问片外 RAM, 也不能读代码区
MOVB	访问片外数据存储器	MOVB A, @DPTR	只能访问片外数据存储器; 地址寄存器只能是 DPTR 或 R0/R1, 不能写成 @R2、@A
MOVC	查表, 访问程序存储器	MOVC A, @A+DPTR	只读代码区; 目标寄存器固定是 A; 地址形成只能是 @A+DPTR 或 @A+PC
PUSH	入栈	PUSH ACC	写法本质是压直接地址对应的内容; 不能写 PUSH R0, 若要压 R0 通常先转存
POP	出栈	POP 30H	出栈顺序与入栈相反; 要注意 SP 初值
XCH	交换数据	XCH A, R0	一端必须是 A; 不能直接 XCH 30H, 31H
ADD	无符号加法	ADD A, #25H	不带进位; 多字节加法应用 ADDC
ADDC	带进位加法	ADDC A, 31H	用前先明确 C 的状态, 通常先 CLR C
SUBB	带借位减法	SUBB A, 32H	多字节减法前一般先 CLR C
INC	加 1	INC R0	不影响进位标志 C; INC DPTR 是 16 位加 1
DEC	减 1	DEC 30H	同样不影响 C; 8051 没有 DEC DPTR 指令
MUL AB	8 位乘法	MUL AB	结果放在 B:A, 不是仍只在 A 中
DIV AB	8 位除法	DIV AB	商在 A, 余数在 B; B=0 时出错
ANL	按位与	ANL A, #0FH	常用于屏蔽位; 注意不是逻辑与短路; ANL C, bit 是位逻辑形式
ORL	按位或	ORL P1, #01H	置位很方便, 但会保留其他位原值; 若想只操作单个位可直接用 SETB
XRL	按位异或	XRL A, #0FFH	可用于逐位取反, 但与 CPL 的对象不同; 不支持所有位寻址形式
CLR	清零位/清累加器/清进位	CLR C	CLR 只能用于 A、C、位地址, 不能写 CLR R0
CPL	取反	CPL P1.0	只能对 A、C 或位地址使用, 不能写 CPL R2
RL / RR	不带进位循环移位	RR A	不经过 C, 与 RLC/RRC 不同
RLC / RRC	带进位循环移位	RRC A	旧 C 会参与移位, 做位统计前要想清楚
SWAP	高低 4 位交换	SWAP A	常用于 BCD 拆分, 只对 A 有效
SETB	置位	SETB TRO	只能对 C 或位地址有效, 不能写 SETB A、SETB R0
JC / JNC	按进位跳转	JC NEXT	依赖 C 当前值, 前序算术或比较会影响它
JZ / JNZ	按 A 是否为 0 跳转	JNZ LOOP	判断对象是累加器 A, 不是任意寄存器; 若判断 R 寄存器常配合 CJNE/DJNZ

指令	用法	举例	易错点
JB / JNB	按位跳转	JB ACC.7,NEG	位地址必须可位寻址；不是所有 RAM 单元都能写成 30H.0 这种位访问
JBC	测位且清位后跳转	JBC TF0,SVC	常用于标志位处理，执行跳转前会自动清该位
CJNE	比较不等则跳转	CJNE A,#34H,N1	比较后会影响 C；若前者小于后者则 C=1
DJNZ	减 1 不为 0 跳转	DJNZ R7,LOOP	很适合循环计数，但不会影响 C；DJNZ 没有立即数形式
SJMP	短跳转	SJMP HERE	位移范围只有 -128 ~ +127 字节
AJMP / ACALL	页内跳转/调用	ACALL DELAY	只能在当前 2KB 页内使用
LJMP / LCALL	长跳转/调用	LJMP START	可跨 64KB 全空间，但代码字节更多
RET	子程序返回	RET	只能用于普通子程序，不用于中断
RETI	中断返回	RETI	中断服务必须用 RETI，不能只用 RET
NOP	空操作	NOP	常用于微调时序，但不要滥用造成代码臃肿

B 附录 B：寄存器位定义与中断定时器配置速查

1. 常用寄存器位定义与含义

寄存器	位名	含义
IE	EA	总中断允许位，EA=1 时 CPU 才响应已开放的中断
IE	ET2	定时器 2 中断允许位（AT89S52 等带 T2 型号）
IE	ES	串口中断允许位
IE	ET1	定时器 1 中断允许位
IE	EX1	外部中断 1 允许位
IE	ET0	定时器 0 中断允许位
IE	EX0	外部中断 0 允许位
IP	PT2	定时器 2 中断优先级位，1 为高优先级
IP	PS	串口中断优先级位
IP	PT1	定时器 1 中断优先级位
IP	PX1	外部中断 1 优先级位
IP	PT0	定时器 0 中断优先级位
IP	PX0	外部中断 0 优先级位
TCON	TF1	定时器 1 溢出标志位，溢出时由硬件置 1
TCON	TR1	定时器 1 运行控制位，1 启动 T1
TCON	TF0	定时器 0 溢出标志位
TCON	TR0	定时器 0 运行控制位，1 启动 T0
TCON	IE1	外部中断 1 请求标志位
TCON	IT1	外部中断 1 触发方式位，1 为边沿触发，0 为低电平触发
TCON	IE0	外部中断 0 请求标志位
TCON	IT0	外部中断 0 触发方式位，1 为边沿触发，0 为低电平触发
TMOD	GATE1/GATE0	门控位，1 时需 TRx=1 且外中断引脚为高电平才运行
TMOD	C/T1、C/T0	0 为定时方式，1 为计数方式
TMOD	M1、M0	定时器工作方式选择位，00/01/10/11 对应方式 0/1/2/3
SCON	SM0、SM1	串口工作方式选择位，四种方式由 SM0/SM1 组合决定
SCON	SM2	多机通信控制位，1 时在方式 2/3 中可只接收地址帧
SCON	REN	串口接收允许位，1 允许接收
SCON	TB8	发送的第 9 位数据
SCON	RB8	接收的第 9 位数据
SCON	TI	发送中断标志位，发送完 1 帧后置 1
SCON	RI	接收中断标志位，接收完 1 帧后置 1
PCON	SMOD	波特率倍增位，1 时串口方式 1/2/3 波特率加倍
T2CON	TF2	定时器 2 溢出标志位
T2CON	EXF2	T2EX 引脚引起的捕获/重装事件标志位
T2CON	RCLK/TCLK	设为 1 时可选用 T2 作为串口接收/发送波特率发生器
T2CON	EXEN2	允许 T2EX 触发捕获/重装控制
T2CON	TR2	定时器 2 启动控制位

寄存器	位名	含义
T2CON	C/T2	0 为定时方式, 1 为计数方式
T2CON	CP/RL2	0 为自动重装, 1 为捕获方式

2. 如何配置中断

基本步骤:

1. 明确中断源: 外部中断、定时器中断、串口中断、定时器 2 中断等。
2. 写中断服务函数:

```
void timer0_isr(void) interrupt 1
{
    /* service code */
}
```

3. 配置相关工作模式:

- 外部中断: 设置 IT0/IT1 为边沿还是电平触发;
- 定时器: 设置 TMOD/T2CON;
- 串口: 设置 SCON 和波特率发生器。

4. 开中断允许位:

- 总中断允许 EA=1
- 分中断允许如 EX0、ET0、ET1、ES、ET2

5. 如有必要, 设置优先级 IP。

IE 寄存器位序:

位名	EA	-	ET2	ES	ET1	EX1	ET0	EX0
----	----	---	-----	----	-----	-----	-----	-----

IP 寄存器位序:

位名	-	-	PT2	PS	PT1	PX1	PT0	PX0
----	---	---	-----	----	-----	-----	-----	-----

例 1: 配置外部中断 0 为下降沿触发:

```
IT0 = 1;
EX0 = 1;
EA = 1;
```

例 2: 配置定时器 0 中断:

```
TMOD &= 0xF0;
TMOD |= 0x01;
TH0 = 0xFC;
TL0 = 0x18;
ET0 = 1;
EA = 1;
TR0 = 1;
```

3. 定时器 T0/T1 的四种模式

模式	特点	典型用途
方式 0	13 位计数器, 容量 8192	老设计、特殊兼容场景, 现已较少用
方式 1	16 位计数器, 容量 65536	最常用, 定时范围大, 适合一般定时中断
方式 2	8 位自动重装, 容量 256	固定周期中断、波特率发生、规则节拍
方式 3	T0 拆成两个独立 8 位计数器	多定时任务但资源紧张时使用; 此时 T1 功能受影响

TMOD 各位定义:

T1				T0			
GATE	C/T	M1	M0	GATE	C/T	M1	M0

说明:

- GATE=0: 仅由 TRx 控制定时器运行。
- GATE=1: 还要求外部中断引脚为高电平。
- C/T=0: 定时方式。
- C/T=1: 计数方式。
- M1M0=00/01/10/11 对应方式 0/1/2/3。

4. 如何计算定时器参数

第 1 步: 求计数节拍

对于传统 12T 8051:

$$T_{\text{tick}} = \frac{12}{f_{\text{osc}}}$$

例如 $f_{\text{osc}} = 12\text{MHz}$, 则:

$$T_{\text{tick}} = 1\mu\text{s}$$

第 2 步: 求所需计数值

$$N = \frac{T_{\text{delay}}}{T_{\text{tick}}}$$

例如需要 1ms :

$$N = \frac{1000\mu\text{s}}{1\mu\text{s}} = 1000$$

第 3 步: 按模式求初值

- 方式 0 (13 位):

$$\text{初值} = 8192 - N$$

- 方式 1 (16 位):

$$\text{初值} = 65536 - N$$

- 方式 2 (8 位自动重装):

$$\text{初值} = 256 - N$$

其中要求 $N \leq 256$ 。

第 4 步: 拆分高低字节

以方式 1 为例, 若初值为 FC18H, 则:

$$TH0 = FCH, \quad TL0 = 18H$$

5. 如何把参数配置到定时器里

例: 12MHz 晶振, T0 方式 1 实现 1ms 定时中断

计算过程:

1. 机器周期: $1\mu s$
2. 计数值: 1000
3. 初值: $65536 - 1000 = 64536 = FC18H$

配置代码:

```
#include <REG52.H>

void timer0_init_1ms(void)
{
    TMOD &= 0xF0;
    TMOD |= 0x01;
    TH0 = 0xFC;
    TL0 = 0x18;
    ET0 = 1;
    EA = 1;
    TR0 = 1;
}

void timer0_isr(void) interrupt 1
{
    TH0 = 0xFC;
    TL0 = 0x18;
}
```

注意点:

- 若使用方式 1 中断, 每次进中断都要重装初值。
- 若使用方式 2, 初值写入 THx 后, TLx 溢出会自动重装, 一般不必每次手动重装。
- 程序中若已占用定时器作串口波特率发生器, 不能随意改动其模式和初值。
- 题目若晶振不是 12MHz, 必须重新计算, 不能照抄 FC18H、FF06H 等常见数值。
- 新型号单片机可能支持 1T 模式, 此时计数节拍不是 $\frac{12}{f_{osc}}$, 要看芯片手册。

6. 四种模式的典型配置模板

方式 0: 13 位

```
TMOD &= 0xF0;  
TMOD |= 0x00;
```

方式 1: 16 位

```
TMOD &= 0xF0;  
TMOD |= 0x01;
```

方式 2: 8 位自动重装

```
TMOD &= 0xF0;  
TMOD |= 0x02;  
TH0 = reload_value;  
TL0 = reload_value;
```

方式 3: T0 分裂模式

```
TMOD &= 0xF0;  
TMOD |= 0x03;
```

说明:

- 方式 3 下, TL0 和 TH0 变成两个独立的 8 位计数器。
- 此时定时器 1 的常规控制功能会受到影响, 因此考试中若无特别需求, 一般优先选方式 1 或方式 2。

7. 考试中最常用的结论

1. 大范围定时: 优先用方式 1。
2. 固定周期重复中断: 优先用方式 2。
3. 多字节加减: 先低字节, 后高字节; 减法前常先 CLR C。
4. 外部中断边沿触发: 设置 IT0=1 或 IT1=1。
5. 串口方式 1 最常考; 11.0592MHz 下常见波特率重装值要会算。